

Universitat Politècnica de Catalunya
Grau en Enginyeria Informàtica
Inteligència Artificial

Sistema experto de recomendación de viajes

Práctica de Sistemas Basados en el Conocimiento

Documentación de la práctica
Curso 2025/2026 – 2Q

Autores

Bernat Llorens
Santiago Fernandez
Roger García

Índice

1. Introducción	1
1.1. Contexto de la práctica y motivación del sistema	1
2. Identificación del problema	1
2.1. Descripción del problema de recomendación de viajes	1
2.2. Alcance, supuestos y límites del sistema	1
2.3. Viabilidad de construir un Sistema Basado en el Conocimiento	2
2.4. Fuentes de conocimiento utilizadas	2
2.5. Objetivos del sistema	3
2.6. Resultados esperados y requisitos de la recomendación	3
3. Conceptualización	4
3.1. Descripción del dominio de viajes personalizados	4
3.2. Conceptos principales del dominio	4
3.2.1. Usuario, perfil de viaje y grupo viajero	4
3.2.2. Ciudades y características turísticas	4
3.2.3. Alojamientos y calidad de estancia	5
3.2.4. Actividades, visitas y guías de visita	5
3.2.5. Conexiones, transportes y etapas del recorrido	5
3.2.6. Restricciones obligatorias y preferencias comparativas	5
3.2.7. Planes candidatos y planes recomendados	6
3.3. Conocimiento experto extraído del dominio	6
3.3.1. Reglas de sentido común sobre perfiles de usuario	6
3.3.2. Criterios de compatibilidad de destinos, alojamientos y transportes	6
3.3.3. Criterios para comparar soluciones factibles	6
3.3.4. Tratamiento de restricciones incompatibles y no solución	7
3.4. Conceptualización apoyada por un modelo de lenguaje	7
3.4.1. Contexto y prompt de elicitación de conocimiento	7
3.4.2. Preguntas realizadas al experto simulado	7
3.4.3. Conocimiento incorporado, adaptado o descartado	8
3.5. Descomposición del problema en subproblemas	8
3.5.1. Adquisición de datos del usuario	8
3.5.2. Deducción del perfil, restricciones y preferencias	8
3.5.3. Filtrado de recursos compatibles	8
3.5.4. Generación de itinerarios candidatos	8
3.5.5. Cálculo de precio y puntuación de alternativas	9
3.5.6. Selección de dos planes con ciudades diferentes	9
3.5.7. Explicación del resultado o del caso sin recomendación	9

3.6.	Descripción informal del proceso de resolución	9
4.	Formalización	9
4.1.	Naturaleza del problema y método de resolución escogido	9
4.1.1.	Clasificación del perfil del usuario	9
4.1.2.	Generación, revisión y comparación de planes	10
4.1.3.	Encaje entre los subproblemas y el método de resolución	10
4.2.	Desarrollo de la ontología	10
4.2.1.	Dominio, cobertura y preguntas que debe responder	10
4.2.2.	Términos relevantes del dominio	11
4.2.3.	Clases, jerarquía y decisiones de modelado	11
4.2.4.	Propiedades, atributos y restricciones de valores	12
4.2.5.	Relaciones entre conceptos	12
4.2.6.	Instancias necesarias para la base de conocimiento	12
4.3.	Ontología final del sistema	12
4.3.1.	Grafo de la jerarquía de conceptos	12
4.3.2.	Descripción detallada de clases	13
4.3.3.	Descripción detallada de atributos y relaciones	13
4.3.4.	Correspondencia entre la ontología y la representación en CLIPS	13
4.4.	Formalización del razonamiento por subproblemas	14
4.4.1.	Datos de entrada y hechos de trabajo	14
4.4.2.	Restricciones duras del viaje	14
4.4.3.	Preferencias y heurísticas de puntuación	15
4.4.4.	Construcción del precio total y duración del viaje	15
4.4.5.	Elección de dos recomendaciones diferentes	15
4.4.6.	Justificación de preferencias cumplidas y no solución	15
5.	Implementación	16
5.1.	Representación del conocimiento en CLIPS	16
5.1.1.	<code>deftemplates</code> y hechos principales	16
5.1.2.	Base de conocimiento de ciudades, alojamientos, visitas y conexiones	16
5.1.3.	Hechos dinámicos de usuario, candidatos y planes elegidos	16
5.2.	Funciones auxiliares del sistema	17
5.3.	Reglas y fases del motor de inferencia	17
5.3.1.	Bienvenida y entrevista	17
5.3.2.	Deducciones expertas	17
5.3.3.	Generación de candidatos	18
5.3.4.	Selección de recomendaciones	18
5.3.5.	Salida explicada y cierre del sistema	18
5.4.	Mecanismos de control y modularización del razonamiento	18
5.5.	Decisiones de implementación relevantes	19

5.5.1.	Restricciones obligatorias frente a preferencias	19
5.5.2.	Cálculo numérico de precios	19
5.5.3.	Control de rutas distintas y caso imposible	19
5.6.	Desarrollo incremental y prototipos	19
5.6.1.	Prototipo inicial	19
5.6.2.	Extensiones y revisiones intermedias	19
5.6.3.	Versión final	20
5.6.4.	Planificación y organización del trabajo del grupo	20
5.7.	Instrucciones de ejecución	20
6.	Juegos de prueba y validación	21
6.1.	Criterios de selección de los juegos de prueba	21
6.2.	Cobertura de casos comunes, restricciones y excepciones	21
6.3.	Caso 1: familia con niños y presupuesto alto	21
6.3.1.	Escenario, objetivo y entrada	21
6.3.2.	Resultado esperado	22
6.3.3.	Salida original de CLIPS	22
6.3.4.	Explicación del resultado obtenido	22
6.4.	Caso 2: estudiante que prioriza tren y evita avión	22
6.4.1.	Escenario, objetivo y entrada	22
6.4.2.	Resultado esperado	22
6.4.3.	Salida original de CLIPS	23
6.4.4.	Explicación del resultado obtenido	23
6.5.	Caso 3: pareja romántica por aniversario	23
6.5.1.	Escenario, objetivo y entrada	23
6.5.2.	Resultado esperado	23
6.5.3.	Salida original de CLIPS	23
6.5.4.	Explicación del resultado obtenido	24
6.6.	Caso 4: grupo de amigos en viaje de fin de curso	24
6.6.1.	Escenario, objetivo y entrada	24
6.6.2.	Resultado esperado	24
6.6.3.	Salida original de CLIPS	24
6.6.4.	Explicación del resultado obtenido	24
6.7.	Caso 5: naturaleza y lugares menos conocidos	25
6.7.1.	Escenario, objetivo y entrada	25
6.7.2.	Resultado esperado	25
6.7.3.	Salida original de CLIPS	25
6.7.4.	Explicación del resultado obtenido	25
6.8.	Caso 6: restricciones incompatibles y no solución	25
6.8.1.	Escenario, objetivo y entrada	25

6.8.2. Resultado esperado	26
6.8.3. Salida original de CLIPS	26
6.8.4. Explicación del resultado obtenido	26
6.9. Verificación del cumplimiento del enunciado	26
6.10. Análisis global de los resultados de prueba	26
7. Conclusiones	26
7.1. Cumplimiento de objetivos	27
7.2. Fortalezas del sistema construido	27
7.3. Limitaciones actuales y posibles mejoras	27
7.4. Aprendizajes sobre ontologías, reglas y Sistemas Basados en el Conocimiento .	27
A. Salidas literales completas de los juegos de prueba	28
A.1. Preparación de la ejecución	28
A.2. Caso familia	28
A.3. Caso estudiante	29
A.4. Caso pareja	29
A.5. Caso amigos	30
A.6. Caso naturaleza	31
A.7. Caso sin solución	32
B. Documentación de la ontología generada con Protégé	32
B.1. Archivo y alcance de la ontología	32
B.2. Grafo de clases y relaciones	32
B.3. Fichas de clases, propiedades e instancias	33
B.3.1. Clases principales	33
B.3.2. Propiedades de objeto	33
B.3.3. Propiedades de datos	34
B.3.4. Instancias representadas en la base de conocimiento	34

1 Introducción

1.1 Contexto de la práctica y motivación del sistema

En esta práctica hemos desarrollado un Sistema Basado en el Conocimiento orientado a recomendar viajes personalizados para la agencia “Al fin del mundo y más allá”. El problema no consiste solamente en escoger un destino bonito: un viaje completo depende del tipo de usuario, de sus restricciones y de varias decisiones que están relacionadas entre sí, como el número de ciudades, los días de estancia, el alojamiento, los transportes entre etapas y las visitas que se proponen en cada ciudad.

Por ejemplo, no tiene sentido tratar igual una escapada de aniversario, un viaje familiar con niños o un grupo de amigos que viaja por fin de curso. Dos usuarios pueden tener el mismo presupuesto y aun así necesitar propuestas muy distintas. Además, algunas condiciones se tienen que cumplir de forma obligatoria, como no superar el presupuesto o evitar el avión si el usuario lo pide, mientras que otras sirven para comparar alternativas, como priorizar el tren o preferir ciudades menos masificadas.

Por este motivo hemos planteado el sistema como un SBC implementado en CLIPS. La idea es representar de forma explícita el conocimiento del dominio y usar reglas para ir pasando de la información que nos da el usuario a dos recomendaciones completas y justificadas. De esta manera no nos limitamos a mostrar una lista fija de viajes, sino que el sistema puede preguntar, deducir, filtrar combinaciones inviables y comparar las soluciones factibles.

2 Identificación del problema

2.1 Descripción del problema de recomendación de viajes

El problema que queremos resolver es la personalización de viajes de varias ciudades a partir de la información de un cliente. El sistema tiene que obtener primero los datos que condicionan la recomendación: edad del viajero principal, tipo de grupo, si viajan niños, si existe un evento especial, objetivo del viaje, presupuesto, duración, número de ciudades, calidad mínima de alojamiento y preferencias sobre transporte o destinos menos conocidos.

Con esos datos se debe construir una recomendación concreta. En nuestro caso, un plan no es solo una ciudad candidata: es un itinerario de dos o tres ciudades con los días repartidos entre ellas, los transportes necesarios para salir del origen, moverse entre etapas y volver, un alojamiento por ciudad, visitas recomendadas y un precio total estimado por persona.

El problema tiene una parte de análisis del usuario y una parte de construcción de la solución. Primero hay que interpretar el perfil y completar algunas decisiones razonables cuando la entrada lo permite. Por ejemplo, una boda o un aniversario apuntan a un viaje romántico, y un viaje corto no debería acabar en una ruta demasiado extensa. Después hay que construir planes que respeten las restricciones y escoger entre las alternativas posibles.

2.2 Alcance, supuestos y límites del sistema

Hemos acotado el alcance para poder razonar con suficiente detalle sobre los planes sin convertir la práctica en un buscador de reservas real. El sistema trabaja con una base de conocimiento cerrada de ciudades, alojamientos, actividades, guías de visita y conexiones de transporte. A partir de esa base genera viajes de dos o tres ciudades y considera un origen genérico para las conexiones de ida y vuelta.

Los precios son estimaciones por persona y se usan para comparar y validar planes dentro del sistema. Para construirlos hemos tomado como referencia la información consultada sobre destinos, alojamientos, visitas y conexiones, pero no pretendemos que la base de conocimiento reproduzca precios de mercado, disponibilidad hotelera o rutas comerciales actualizadas. En cambio, sí hemos intentado mantener criterios coherentes: los transportes lejanos son más caros, la calidad del alojamiento influye en el precio y un viaje corto limita las combinaciones razonables.

El sistema tampoco intenta resolver todos los detalles de la organización real de un viaje. No gestiona fechas concretas, horarios, visados, meteorología, reservas ni incidencias durante el recorrido. Su objetivo es producir recomendaciones explicables a partir del conocimiento que hemos representado, y detectar cuando las restricciones del usuario no dejan ninguna alternativa factible con la base disponible.

2.3 Viabilidad de construir un Sistema Basado en el Conocimiento

Consideramos que el problema es adecuado para construir un SBC porque no basta con aplicar una fórmula sobre los datos del usuario. Para recomendar un viaje hay que combinar conocimiento del dominio con decisiones de experto: distinguir restricciones de preferencias, interpretar algunos perfiles, descartar combinaciones poco razonables y comparar rutas que ya son factibles.

Además, el problema se puede describir mediante objetos y relaciones claras. Tenemos usuarios, ciudades, alojamientos, actividades, conexiones y planes. Sobre estos conceptos aparecen reglas bastante naturales: si se viaja con niños se deben evitar recursos no aptos, si se quiere evitar el avión se descartan rutas que lo usan, si el presupuesto no cubre el precio calculado el plan no sirve, y si el usuario prefiere tren esa característica puede mejorar la puntuación de una alternativa sin convertirla en obligatoria.

Esta separación entre conocimiento representado y reglas de razonamiento nos permite justificar las recomendaciones y también explicar por qué puede no haber solución. Por eso un lenguaje de reglas como CLIPS encaja bien con el problema: el sistema puede ir acumulando hechos sobre el usuario y los planes, activar reglas cuando se cumplen sus condiciones y controlar el razonamiento por fases hasta llegar a la salida.

2.4 Fuentes de conocimiento utilizadas

Para construir el sistema hemos usado varias fuentes de conocimiento. La primera ha sido el propio enunciado de la práctica, que fija el problema que hay que resolver, los datos relevantes que se deben considerar y la diferencia entre restricciones obligatorias y preferencias que permiten comparar soluciones.

También hemos usado el material de la asignatura sobre representación del conocimiento, ontologías y arquitectura de los SBC. Ese material nos ha servido para estructurar el dominio, construir la ontología, separar la base de conocimiento del razonamiento y organizar la documentación siguiendo las fases de la ingeniería del conocimiento.

Para la parte concreta de viajes hemos consultado información turística de catálogos y bases de datos de agencias de viajes, alojamientos, actividades y transportes. A partir de esa información hemos seleccionado destinos, alojamientos, visitas y conexiones que nos permitieran construir una base de conocimiento variada, y los hemos adaptado al nivel de detalle que necesita el sistema con precios estimados, niveles de calidad, popularidad y etiquetas turísticas. El objetivo de esos datos no es sustituir a una agencia real, sino representar suficiente conocimiento del dominio para que las reglas produzcan itinerarios distintos y casos de prueba relevantes.

Por último, durante la conceptualización hemos usado un modelo de lenguaje como experto simulado para contrastar criterios de recomendación. Esa conversación nos ha servido como apoyo para decidir qué conocimiento del dominio era útil convertir en reglas o preferencias. Documentaremos el prompt, las preguntas relevantes y el conocimiento que hemos aprovechado en el apartado específico de conceptualización.

2.5 Objetivos del sistema

Los objetivos que nos hemos marcado para el sistema son los siguientes:

- Obtener del usuario la información necesaria para personalizar el viaje y no basar la recomendación solo en el presupuesto o en un destino aislado.
- Deducir parte del perfil del viajero y aplicar conocimiento de sentido común cuando las respuestas permiten completar la interpretación del caso.
- Representar de forma explícita las restricciones del viaje y descartar cualquier plan que no las cumpla.
- Tratar las preferencias como criterios de comparación para puntuar soluciones factibles y escoger alternativas más adecuadas.
- Generar viajes multi-ciudad con alojamiento, transportes, visitas, duración y precio total estimado.
- Recomendar dos planes diferentes cuando la base de conocimiento lo permita, e informar claramente si no existe ninguna recomendación factible.

2.6 Resultados esperados y requisitos de la recomendación

Al final de la ejecución esperamos que el usuario reciba dos recomendaciones de viaje suficientemente concretas para entender qué propone el sistema y por qué. Cada plan debe indicar:

- el precio total estimado por persona;
- la duración total del viaje;
- las ciudades visitadas y los días asignados a cada una;
- las visitas recomendadas en cada ciudad;
- el alojamiento escogido en cada etapa;
- los transportes desde el origen, entre ciudades y de vuelta;
- las preferencias del usuario que se han cumplido.

Las dos recomendaciones tienen que ser alternativas reales y no dos impresiones del mismo plan. Por eso se busca que visiten ciudades diferentes y se seleccionan sobre el conjunto de candidatos que ya cumplen presupuesto, duración, calidad mínima y restricciones de transporte. Si las condiciones del usuario son incompatibles con la base de conocimiento disponible, el resultado esperado pasa a ser un mensaje explícito de no solución en lugar de forzar una recomendación incorrecta.

3 Conceptualización

3.1 Descripción del dominio de viajes personalizados

En la fase de conceptualización hemos pasado del enunciado general de “recomendar viajes” a una descripción del dominio que el sistema pueda tratar. Para nosotros un viaje personalizado es una propuesta que relaciona dos partes: por un lado el cliente, con sus condiciones y sus gustos; por el otro los recursos turísticos disponibles, que son destinos, alojamientos, visitas y transportes.

La recomendación no se decide con un único dato. Un presupuesto alto no convierte cualquier ruta en buena, igual que escoger un motivo cultural no anula la duración, el tipo de grupo o la calidad mínima de alojamiento. Por eso el dominio mezcla restricciones que se deben cumplir con criterios que permiten escoger la alternativa más adecuada entre varias que ya son factibles.

El resultado del dominio tampoco es un destino aislado. La solución que queremos construir es un itinerario de dos o tres ciudades con etapas ordenadas. Cada etapa necesita una ciudad, unos días de estancia, un alojamiento y unas visitas concretas; además el plan completo necesita los transportes de ida, entre ciudades y de vuelta, un precio estimado y una explicación breve de las preferencias que satisface.

Con esta forma de ver el problema, el conocimiento que necesitamos no se limita a almacenar datos de turismo. También hace falta representar decisiones de sentido común que usaría una agencia al hablar con un cliente: qué destinos encajan mejor con un objetivo romántico o de descanso, qué cambia cuando viajan niños, cuándo una ruta lejana deja de tener sentido para pocos días y cómo comparar dos viajes que cumplen las condiciones obligatorias.

3.2 Conceptos principales del dominio

3.2.1 Usuario, perfil de viaje y grupo viajero

El punto de partida es el usuario. No lo tratamos solo como la persona que ejecuta el programa, sino como el viajero o grupo para el que se prepara la recomendación. De él nos interesan la edad del viajero principal, la composición del grupo, si viajan niños y si el viaje está ligado a un evento especial como una boda, un aniversario o un fin de curso.

Sobre el usuario también aparecen las condiciones propias del viaje: presupuesto máximo, duración disponible, número de ciudades que se quiere visitar, calidad mínima del alojamiento y decisiones sobre el transporte. A eso se añaden preferencias más blandas, por ejemplo querer usar tren cuando sea razonable o preferir ciudades menos masificadas.

El perfil de viaje es la lectura que hacemos de esa información. El usuario puede indicar directamente su motivo principal, pero hay casos donde el sistema puede completar la interpretación: una boda o un aniversario apuntan a un viaje romántico, un fin de curso a uno de diversión y un grupo con niños obliga a pensar en destinos y alojamientos adecuados. Esta separación entre datos preguntados y perfil interpretado es importante porque es donde aparece parte del conocimiento experto.

3.2.2 Ciudades y características turísticas

Las ciudades son los destinos que pueden formar parte de una ruta. Para recomendarlas no basta con conocer su nombre. Necesitamos saber su país, la zona o continente, un coste diario estimado, su nivel de seguridad, si son aptas para viajar con niños, su popularidad y un conjunto de etiquetas turísticas.

Las etiquetas resumen qué puede aportar cada destino: cultural, romántico, descanso, diversión, naturaleza, playa, gastronomía o un carácter más icónico. Gracias a ellas se puede razonar sobre la compatibilidad entre el objetivo del viaje y la ciudad sin convertir el sistema en una lista cerrada de respuestas prefijadas para cada caso.

3.2.3 Alojamientos y calidad de estancia

Un alojamiento es un recurso asociado a una ciudad. En el dominio nos interesan su nombre, categoría, calidad, precio por noche y compatibilidad con niños. Estos datos permiten cubrir dos necesidades diferentes: comprobar que se respeta la calidad mínima que exige el usuario y construir el precio total del plan con un coste de estancia concreto.

La calidad del alojamiento se trata con cuidado porque puede actuar como restricción o como margen de ajuste. Si el usuario pide una calidad mínima, el viaje no debería bajar de ella sin permiso. En cambio, cuando declara que acepta sacrificar calidad para ajustar presupuesto, se puede permitir una flexibilización controlada en vez de inventar una solución que no existe.

3.2.4 Actividades, visitas y guías de visita

Las actividades representan visitas concretas que se pueden hacer en una ciudad. Cada una tiene nombre, tipo, duración, coste, prioridad y compatibilidad familiar. Nos sirven para que la base de conocimiento no se quede en frases genéricas del estilo “hacer turismo por la ciudad”.

Para la recomendación final usamos además guías de visita. Una guía agrupa visitas adecuadas para una ciudad y un motivo de viaje, indica el mínimo de días que necesita y aporta un coste total de esas visitas. De esta manera, la etapa de una ciudad ya lleva una propuesta concreta distinta si el usuario viaja por cultura, descanso, naturaleza, diversión o romanticismo.

3.2.5 Conexiones, transportes y etapas del recorrido

Las conexiones expresan qué transportes están disponibles entre el origen y una ciudad o entre dos ciudades de la ruta. De cada conexión necesitamos el medio, el coste, la duración aproximada y el ámbito del tramo. El ámbito permite distinguir, por ejemplo, un salto intercontinental de un desplazamiento local o europeo.

Una etapa es la parte del viaje que ocurre en una ciudad. Aunque en la implementación final la etapa queda embebida dentro del plan candidato, en la conceptualización nos ayuda a ordenar la solución: una ruta está formada por etapas y conexiones entre ellas.

3.2.6 Restricciones obligatorias y preferencias comparativas

Una decisión central de la conceptualización ha sido separar restricciones de preferencias. Las restricciones son las condiciones que no queremos violar al generar una recomendación: no superar el presupuesto, respetar la duración disponible, aceptar solo el número de ciudades permitido, evitar el avión cuando se pide, mantener la calidad mínima y exigir compatibilidad familiar cuando viajan niños.

Las preferencias no tienen el mismo papel. Se usan para comparar soluciones que ya han pasado los filtros obligatorios. Por ejemplo, una ruta puede recibir mejor valoración si usa tren cuando el usuario lo prefiere, si incluye ciudades menos conocidas, si mejora la calidad mínima pedida o si deja margen de presupuesto. Así evitamos descartar un viaje válido solo porque no cumple un gusto opcional.

3.2.7 Planes candidatos y planes recomendados

Un plan candidato es una combinación que el sistema ha conseguido construir con ciudades, días, alojamientos, transportes, visitas, precio y puntuación. Solo llega a ser candidato cuando respeta las restricciones que se han comprobado durante la generación.

Un plan recomendado es uno de los candidatos que se escoge al final. El sistema no debe quedarse con el primero que aparezca, sino comparar las alternativas disponibles y elegir dos rutas distintas siempre que la base de conocimiento lo permita. Esta diferencia entre candidato y recomendación nos permite separar la construcción de soluciones de la elección final.

3.3 Conocimiento experto extraído del dominio

3.3.1 Reglas de sentido común sobre perfiles de usuario

Durante la conceptualización hemos identificado reglas sencillas que no son datos turísticos puros, sino decisiones que usaría una persona que prepara viajes. Algunos ejemplos que hemos incorporado son:

- Una boda o un aniversario se interpreta como una situación donde tiene sentido priorizar un perfil romántico.
- Un fin de curso encaja mejor con un perfil de ocio y diversión.
- Si viajan niños, hay que dar importancia a destinos y alojamientos aptos y evitar propuestas que dependan de recursos no familiares.
- Si un usuario que viaja solo no declara un motivo claro, una ruta cultural es un punto de partida razonable y explicable.

Estas reglas no sustituyen las respuestas del usuario cuando ya son claras. Sirven para completar información poco concreta y para que el sistema pueda seguir razonando con un perfil de viaje definido.

3.3.2 Criterios de compatibilidad de destinos, alojamientos y transportes

El siguiente bloque de conocimiento experto consiste en decidir qué recursos pueden convivir dentro de una misma recomendación. En destinos se comprueba que el motivo del viaje encaje con las etiquetas de las ciudades. En alojamientos se comprueba la calidad solicitada y, cuando hay niños, que el recurso sea apto. En las visitas se escoge una guía coherente con la ciudad, el objetivo del viaje y los días asignados.

Con los transportes aplicamos criterios parecidos. Si el usuario quiere evitar avión, una ruta que lo necesita deja de ser compatible. Además, una ruta muy lejana no es buena candidata para una escapada demasiado corta: aunque exista una conexión, el tiempo disponible cambia lo que es razonable recomendar.

3.3.3 Criterios para comparar soluciones factibles

Una vez un plan cumple las condiciones obligatorias, todavía puede ser mejor o peor que otro. Para compararlos hemos identificado varios criterios:

- La adecuación base de las ciudades que forman la ruta.

- El uso del tren cuando el usuario lo ha marcado como preferencia.
- La presencia de ciudades menos conocidas o menos masificadas si se han pedido.
- La calidad extra del alojamiento por encima del mínimo solicitado.
- El margen que queda respecto al presupuesto máximo.

Este conocimiento se traduce después en una puntuación heurística, pero en esta fase lo importante es la idea: primero se garantizan las restricciones y después se ordenan las alternativas usando preferencias.

3.3.4 Tratamiento de restricciones incompatibles y no solución

También hemos considerado el caso en el que el cliente pide una combinación que la base de conocimiento no puede satisfacer. Por ejemplo, un presupuesto muy bajo junto con una calidad alta, pocos días y transportes muy limitados puede dejar el conjunto de candidatos vacío.

En ese caso el comportamiento correcto no es relajar condiciones a escondidas ni imprimir un viaje que no cumple lo pedido. La conclusión del razonamiento debe ser que no se ha encontrado una recomendación factible y conviene indicar qué tipo de restricción podría revisarse.

3.4 Conceptualización apoyada por un modelo de lenguaje

3.4.1 Contexto y prompt de elicitación de conocimiento

Como apoyo a esta fase hemos usado un modelo de lenguaje en el papel de experto simulado en recomendación de viajes. No le pedimos que escribiera la ontología ni las reglas de CLIPS, sino que respondiera como lo haría un agente de viajes al explicar sus criterios a un ingeniero del conocimiento. El contexto base utilizado fue el siguiente:

You are going to help a knowledge engineer perform knowledge elicitation for building a rule-based expert system for touristic travel recommendation. You are an expert travel agent. Explain the criteria you use to match traveller profiles, constraints and preferences with destinations, accommodation, transport and activities. Answers must be direct and useful for building rules.

Con este contexto buscábamos obtener conocimiento del dominio y del proceso de decisión, no una solución ya programada. Después contrastamos las ideas útiles con el alcance del enunciado y con lo que realmente queríamos representar en nuestro sistema.

3.4.2 Preguntas realizadas al experto simulado

Las preguntas más relevantes que usamos para orientar la extracción de conocimiento fueron:

- *When travelling with children, what kind of accommodation and destinations are safer?*
- *When the budget is low, is it better to reduce days, quality or transportation cost?*
- *For a romantic trip, which city features matter most?*
- *When should a short trip avoid far-away cities?*
- *How do you compare two valid routes when both satisfy hard constraints?*

Las preguntas están planteadas para sacar criterios de decisión: qué datos del usuario importan, qué condiciones son obligatorias, cuándo se puede flexibilizar algo y qué elementos sirven para comparar rutas posibles.

3.4.3 Conocimiento incorporado, adaptado o descartado

De esa elicitación nos quedamos con varias ideas que encajaban con la práctica. Incorporamos la importancia de la compatibilidad familiar cuando viajan niños, la lectura romántica de viajes de pareja ligados a boda o aniversario, la limitación de rutas lejanas en viajes cortos y la separación entre condiciones duras y preferencias.

Otras ideas se adaptaron al tamaño del sistema. Por ejemplo, el experto puede hablar de ajustar muchos gastos distintos, pero nosotros hemos representado el presupuesto con precios estimados de transporte, estancia, coste diario y visitas, y solo flexibilizamos la calidad del alojamiento cuando el usuario lo permite. También convertimos la comparación informal entre rutas en criterios concretos que luego pueden puntuar planes factibles.

Por último, dejamos fuera detalles que podrían ser realistas en una agencia pero no forman parte de esta base de conocimiento: disponibilidad de reservas, fechas exactas, climatología, documentación de entrada a países o cambios de precio en tiempo real. Incluirlos habría ampliado el dominio sin mejorar la resolución del problema que se pide en la práctica.

3.5 Descomposición del problema en subproblemas

3.5.1 Adquisición de datos del usuario

El primer subproblema es recoger la información necesaria del cliente. Se pregunta por su grupo, niños, evento, motivo, presupuesto, duración, número de ciudades, preferencias de transporte, calidad mínima, posibles sacrificios para ajustar coste, interés por ciudades menos conocidas y ritmo del viaje. Sin esta parte no hay un caso concreto sobre el que razonar.

3.5.2 Deducción del perfil, restricciones y preferencias

Con los datos anteriores se completa el perfil de viaje y se organizan las decisiones que afectarán al resto del proceso. Aquí se deduce el motivo si falta o si un evento lo aclara, se traduce el presupuesto a un nivel orientativo y se fijan las restricciones y preferencias que tendrán un papel distinto en la generación.

3.5.3 Filtrado de recursos compatibles

Antes de construir planes completos hay que evitar combinaciones que ya se sabe que no encajan. Se filtran ciudades por motivo y compatibilidad familiar, alojamientos por calidad y aptitud para niños, guías por motivo y días disponibles, y conexiones por las restricciones de transporte y distancia.

3.5.4 Generación de itinerarios candidatos

Con los recursos compatibles se proponen itinerarios de dos o tres ciudades. Cada candidato reparte los días de estancia entre sus etapas, incorpora los transportes necesarios, selecciona alojamientos y arrastra las visitas que se mostrarán al final. Este subproblema construye el espacio de soluciones sobre el que luego se decidirá.

3.5.5 Cálculo de precio y puntuación de alternativas

Cada itinerario necesita un precio total estimado por persona. Este precio se forma con transportes, coste de alojamientos, coste diario de las ciudades y coste de visitas. Si queda por encima del presupuesto, el candidato no es válido. Si lo respeta, se calcula una puntuación para reflejar su adecuación y las preferencias opcionales que satisface.

3.5.6 Selección de dos planes con ciudades diferentes

Una vez generados los candidatos se escoge el mejor valorado y se busca una segunda alternativa que no sea una copia con otro alojamiento o con una variación irrelevante. La prioridad es encontrar una ruta sin solape de ciudades; si la base no permite tanto, al menos debe ser una ruta distinta.

3.5.7 Explicación del resultado o del caso sin recomendación

El último subproblema es comunicar la conclusión. Cuando hay planes, se imprimen de forma completa con ciudades, días, visitas, alojamientos, transportes, precio, puntuación y preferencias cumplidas. Cuando no aparece ningún candidato factible, se explica que las restricciones no se pueden satisfacer con la base de conocimiento actual en vez de ocultar el fallo.

3.6 Descripción informal del proceso de resolución

Visto de principio a fin, el sistema empieza hablando con el usuario para convertir una petición abierta en un caso concreto. Después interpreta ese caso y completa decisiones sencillas de perfil. Con esa información recorre la base de conocimiento para proponer rutas que tengan ciudades compatibles, transportes posibles, alojamientos aceptables y visitas coherentes.

Cada ruta propuesta se revisa con las restricciones obligatorias. Las que no entran en presupuesto, no caben en los días disponibles o incumplen una condición marcada por el usuario no llegan a la salida. Sobre las rutas que sí sobreviven se aplican criterios de comparación para ordenar alternativas. Finalmente se presentan dos recomendaciones distintas si existen; en caso contrario se informa de que el conjunto de soluciones factibles ha quedado vacío.

4 Formalización

4.1 Naturaleza del problema y método de resolución escogido

4.1.1 Clasificación del perfil del usuario

La primera parte del problema encaja con una tarea de **clasificación**. El sistema recibe datos directos del usuario, pero para recomendar necesita trabajar con un perfil de viaje ya interpretado. El atributo más importante de esa clasificación es el motivo del viaje, que puede quedar como descanso, cultural, diversión, romántico, naturaleza o mixto.

Cuando el usuario ya expresa un motivo claro, esa respuesta se conserva. Si la respuesta queda abierta, la clasificación se apoya en el tipo de grupo y en el evento: una boda o un aniversario orientan el caso hacia el romanticismo, un fin de curso hacia la diversión, viajar con niños hacia una propuesta de descanso y viajar solo sin objetivo explícito hacia una ruta cultural. También se normalizan otros datos antes de generar rutas, como pasar el presupuesto a un nivel orientativo y convertir la calidad mínima del alojamiento a una escala numérica.

Esta primera tarea reduce la ambigüedad de la petición. En vez de generar planes con respuestas incompletas, el sistema llega a la siguiente fase con un usuario formalizado mediante atributos que pueden aparecer en condiciones de reglas.

4.1.2 Generación, revisión y comparación de planes

La segunda parte del problema encaja con un proceso de **propuesta y revisión**. A partir del perfil ya clasificado se proponen combinaciones de recursos de la base de conocimiento: ciudades, alojamientos, guías de visita y conexiones. Cada combinación intenta formar un itinerario de dos o tres ciudades.

La revisión se hace comprobando restricciones durante la propia generación. No se debe aceptar una propuesta si usa recursos incompatibles con niños, si incumple la calidad mínima, si necesita un avión que el usuario ha rechazado, si la distancia no tiene sentido para los días disponibles o si el precio calculado supera el presupuesto. Por eso en la implementación solo se materializan como **PlanCandidato** las combinaciones que han sobrevivido a esos filtros.

El método se completa con una **comparación heurística**. Las restricciones determinan qué planes son válidos; la puntuación permite ordenar los válidos para preferir rutas con mejor adecuación base, tren si se ha pedido, ciudades menos conocidas, calidad por encima del mínimo y margen de presupuesto.

4.1.3 Encaje entre los subproblemas y el método de resolución

La descomposición del apartado anterior se puede encajar en el método de resolución de esta forma:

- La adquisición de datos prepara el hecho que representa al usuario.
- La deducción del perfil aplica la clasificación y completa los atributos que se usarán para decidir.
- El filtrado y la generación de itinerarios forman la parte de propuesta y revisión de soluciones.
- El cálculo de precio revisa la restricción de presupuesto y la puntuación compara los planes factibles.
- La selección final escoge dos recomendaciones distintas a partir del conjunto de candidatos ordenados.
- La salida muestra la solución completa o informa de no solución si la revisión ha dejado el conjunto de candidatos vacío.

Por tanto, no hemos formalizado el sistema como una única regla enorme. La resolución se reparte en subtareas que transforman datos de usuario en hechos de trabajo, hechos candidatos y, finalmente, recomendaciones.

4.2 Desarrollo de la ontología

4.2.1 Dominio, cobertura y preguntas que debe responder

La ontología se ha construido para cubrir el conocimiento que hace falta al recomendar viajes personalizados. El dominio no es el turismo completo, sino la parte que interviene en nuestros

planes: perfil del cliente, destinos, servicios, restricciones, preferencias, etapas y paquete de viaje resultante.

Para decidir la cobertura partimos de preguntas que el sistema debe poder responder:

- Qué información del usuario condiciona la recomendación.
- Qué ciudades, alojamientos, actividades y transportes pueden formar una ruta.
- Qué recursos pertenecen a cada etapa y qué etapas pertenecen al paquete completo.
- Qué condiciones debe respetar un viaje y qué preferencias puede cumplir sin tratarlas como obligatorias.
- Qué precio y qué puntuación resumen la recomendación final.

A partir de esas preguntas hemos seguido un desarrollo progresivo: primero identificamos los conceptos del dominio, después separamos servicios y soluciones, luego añadimos atributos medibles y por último definimos las relaciones necesarias entre el paquete, sus etapas y los recursos usados.

4.2.2 Términos relevantes del dominio

Los términos que hemos considerado relevantes para pasar de la conceptualización a la ontología son:

- Cliente o usuario, grupo viajero, edad, presupuesto, duración, objetivo, restricciones y preferencias.
- Ciudad o destino, zona, coste diario, seguridad, popularidad y etiquetas turísticas.
- Alojamiento, categoría, calidad, precio por noche y compatibilidad familiar.
- Actividad o visita, transporte o conexión y coste asociado.
- Etapa, paquete de viaje, precio total y puntuación.

Estos términos no se han convertido todos en clases. Algunos son atributos, otros son relaciones y otros se concretan como valores controlados en CLIPS. La distinción evita que la ontología tenga clases innecesarias para cada valor de una preferencia o de una categoría.

4.2.3 Clases, jerarquía y decisiones de modelado

La jerarquía parte de la clase general **Entidad**. Bajo ella situamos el usuario, la ciudad, la restricción, la preferencia, el paquete de viaje, la etapa y **Servicio**. Esta última clase agrupa los recursos que se consumen dentro del recorrido. Sus especializaciones son **Alojamiento**, **Transporte** y **Actividad**.

Esta separación responde a dos decisiones de modelado. Primero, el cliente y la solución no son servicios: el usuario describe el caso y el paquete es el resultado. Segundo, restricciones y preferencias se mantienen como conceptos del dominio porque tienen papeles diferentes en la recomendación, aunque en la implementación operativa algunas se representen como atributos del usuario.

Hemos incluido **Etapa** porque un paquete de varias ciudades se entiende mejor como una secuencia de partes: cada etapa visita una ciudad y usa un alojamiento y unas actividades. En CLIPS esta estructura se compacta dentro del plan por simplicidad, pero en la ontología permite expresar el dominio de forma más natural.

4.2.4 Propiedades, atributos y restricciones de valores

Los atributos de datos recogen la información que se necesita medir o mostrar. En el usuario se representan la edad, el presupuesto en euros y la duración máxima. En ciudad aparece el coste diario. En alojamiento aparecen el precio por noche y la calidad. En transporte se guardan coste y horas. Por último, el paquete de viaje tiene precio total y puntuación.

Además de esas propiedades de la ontología, la formalización operativa necesita valores controlados para poder escribir reglas precisas. Por eso en CLIPS se restringen, entre otros, los tipos de grupo, los motivos del viaje, los niveles de calidad, los medios de transporte, la popularidad de una ciudad y los ámbitos de una conexión. Así evitamos que el razonamiento dependa de texto libre.

4.2.5 Relaciones entre conceptos

Las relaciones de objeto de la ontología expresan cómo se arma una recomendación:

- `viajaA` une un `PaqueteViaje` con las `Ciudad` que visita.
- `incluyeEtapa` une el paquete con cada `Etapa`.
- `usaAlojamiento` e `incluyeActividad` describen los recursos de una etapa.
- `usaTransporte` indica los desplazamientos necesarios para el paquete completo.
- `cumplePreferencia` y `respetaRestriccion` distinguen los criterios opcionales de las condiciones obligatorias.

Con estas relaciones la ontología no solo enumera conceptos, sino que permite leer una recomendación como un objeto compuesto.

4.2.6 Instancias necesarias para la base de conocimiento

Para resolver casos concretos necesitamos instancias de los conceptos anteriores. En nuestra base de conocimiento se materializan como hechos iniciales de CLIPS:

- Ciudades disponibles con coste, popularidad, compatibilidad familiar, puntuación base y etiquetas.
- Alojamientos asociados a una ciudad con calidad y precio por noche.
- Actividades y guías de visita que dan contenido concreto a cada etapa.
- Conexiones desde el origen y entre ciudades con medio, coste, horas y ámbito.

Las instancias se han seleccionado a partir de la información de viajes consultada y se han adaptado a la granularidad del sistema. La ontología define su estructura; la base de hechos aporta los recursos concretos con los que el motor puede construir rutas.

4.3 Ontología final del sistema

4.3.1 Grafo de la jerarquía de conceptos

La Figura 1 muestra la jerarquía final de clases. La rama de `Servicio` agrupa los recursos utilizados por el paquete, mientras que las demás ramas representan el caso, la solución y los criterios que se aplican sobre ella.

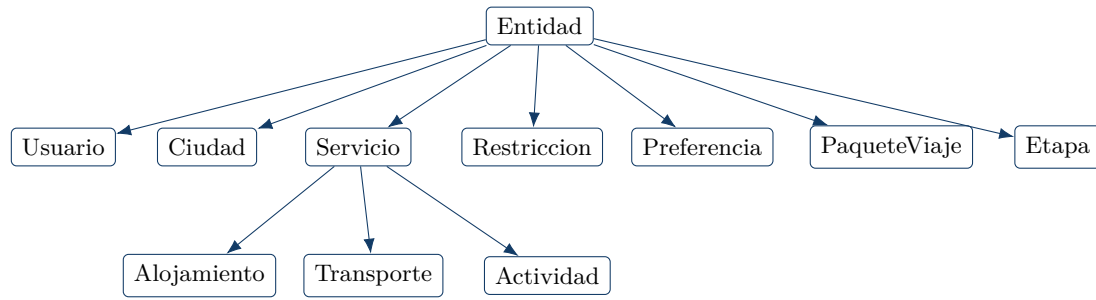


Figura 1: Jerarquía de conceptos de la ontología de viajes.

4.3.2 Descripción detallada de clases

Las clases finales tienen el siguiente papel:

- **Entidad:** raíz común de los conceptos principales del dominio.
- **Usuario:** perfil del cliente, con datos del grupo, objetivo, condiciones y preferencias.
- **Ciudad:** destino candidato que aporta características turísticas y costes.
- **Servicio:** clase general para recursos que se usan dentro de una recomendación.
- **Alojamiento:** servicio de estancia asociado a una ciudad.
- **Transporte:** servicio que permite conectar origen y destinos.
- **Actividad:** visita concreta que puede formar parte de una etapa.
- **Restriccion:** condición que un paquete debe respetar para ser válido.
- **Preferencia:** criterio opcional que mejora o explica una alternativa factible.
- **PaqueteViaje:** solución recomendada con ruta, precio y puntuación.
- **Etapa:** parte del paquete centrada en una ciudad concreta.

4.3.3 Descripción detallada de atributos y relaciones

Las propiedades de datos de la ontología son `edad`, `presupuestoEuros` y `duracionMaxDias` para el usuario; `nombre` para nombrar recursos; `costeDiario` para ciudad; `precioNoche` y `calidad` para alojamiento; `costeTransporte` y `horasTransporte` para transporte; y `precioTotal` y `puntuacion` para el paquete de viaje.

Las relaciones finales conectan el paquete con la ruta y sus recursos. El paquete usa `viajaA`, `incluyeEtapa` y `usaTransporte`; cada etapa usa `usaAlojamiento` e `incluyeActividad`. A ellas se suman `cumplePreferencia` y `respetoRestriccion`, que dejan claro que un paquete se evalúa con dos tipos de criterios.

4.3.4 Correspondencia entre la ontología y la representación en CLIPS

La ontología y CLIPS representan el mismo dominio con niveles de detalle distintos. La correspondencia principal es:

- **Usuario, Ciudad, Alojamiento y Actividad** se convierten directamente en `deftemplate` con el mismo papel.

- **Transporte** se implementa como **Conexion**, porque en el razonamiento interesa guardar origen, destino, medio, coste, horas y ámbito del tramo.
- **PaqueteViaje** se representa con **PlanCandidato**; al seleccionar una recomendación aparece además el hecho **PlanElegido**.
- **Etapas** se aplanan dentro del plan con los campos de ciudad, días, alojamiento, visitas y transporte de cada posición de la ruta.
- **Restriccion** y **Preferencia** se materializan sobre todo como atributos del usuario y como condiciones o puntuaciones en las reglas.

CLIPS añade algunos hechos auxiliares que no son clases del dominio ontológico. **Fase** controla el avance del razonamiento, **DecisionExperta** registra deducciones realizadas y **GuiaVisitas** empaqueta visitas compatibles para que la recomendación final sea concreta.

4.4 Formalización del razonamiento por subproblemas

4.4.1 Datos de entrada y hechos de trabajo

El caso de entrada se formaliza en un hecho **Usuario**. La entrevista rellena sus atributos con respuestas validadas: edad, grupo, presencia de niños, evento, motivo, presupuesto, duración, rango de ciudades, preferencias de transporte, calidad mínima, posibles sacrificios, interés por destinos menos conocidos y ritmo.

El razonamiento combina ese hecho con la base estática de ciudades, alojamientos, actividades, guías de visita y conexiones. Como hechos de trabajo se usan **Fase** para controlar en qué subproblema estamos, **DecisionExperta** para dejar constancia de algunas deducciones, **PlanCandidato** para las soluciones construidas y **PlanElegido** para las recomendaciones finales.

4.4.2 Restricciones duras del viaje

Las restricciones duras se aplican antes de aceptar un plan candidato:

- El número de ciudades debe estar dentro del rango pedido y cada etapa debe respetar los días mínimos y máximos manejados por el sistema.
- El motivo del viaje debe ser compatible con las etiquetas de las ciudades y con la guía de visitas escogida.
- Si viajan niños, ciudades y alojamientos deben estar marcados como aptos.
- La calidad del alojamiento debe alcanzar el mínimo solicitado, salvo la relajación controlada cuando el usuario acepta sacrificar calidad.
- Si el usuario evita avión, ningún tramo de la ruta puede usarlo. En viajes cortos también se descartan tramos intercontinentales poco razonables.
- El precio total calculado debe ser menor o igual que el presupuesto disponible.

Estas condiciones se formalizan como pruebas de compatibilidad y condiciones numéricas durante la generación de rutas.

4.4.3 Preferencias y heurísticas de puntuación

Las preferencias se formalizan como bonificaciones sobre planes que ya son válidos. De forma resumida, la puntuación de una ruta se calcula como:

$$S = \sum p_{base} + B_{tren} + B_{oculto} + B_{calidad} + B_{precio}.$$

El bonus de tren solo aparece si el usuario lo prefiere y algún tramo lo usa. El bonus de ciudades menos conocidas depende de la popularidad de los destinos. El bonus de calidad premia alojamientos que superan la calidad mínima. El bonus de precio favorece que el plan deje margen respecto al presupuesto. Así el sistema puede comparar soluciones sin transformar cada preferencia en una prohibición.

4.4.4 Construcción del precio total y duración del viaje

La duración de un candidato parte de los días indicados por el usuario y se reparte entre sus etapas. En rutas de dos ciudades se divide en dos bloques; en rutas de tres ciudades se distribuye en tres partes. Después se comprueba que cada parte respeta los límites de días por ciudad y el mínimo exigido por la guía de visitas escogida.

El precio total se formaliza sumando transportes, estancias, coste diario de cada ciudad y visitas. Para dos ciudades:

$$P = T + d_1(a_1 + c_1) + d_2(a_2 + c_2) + v_1 + v_2.$$

En esa expresión, T suma los transportes, a_i es el precio por noche del alojamiento de la etapa i , c_i su coste diario y v_i el coste de sus visitas. Para tres ciudades se añade el tercer tramo interno, la tercera estancia y las visitas de la tercera etapa. Esta expresión permite comparar todos los planes con el presupuesto del mismo modo.

4.4.5 Elección de dos recomendaciones diferentes

La selección parte de los `PlanCandidato` ya puntuados. El primer plan elegido es el candidato con mayor score. Para el segundo se prioriza el mejor plan cuya ruta no comparte ciudades con el primero. Si no existe una alternativa sin solape, se busca la mejor ruta distinta, evitando devolver dos veces el mismo recorrido con una variación superficial.

Esta formalización responde a la necesidad de entregar dos alternativas reales. La selección no vuelve a calcular los planes, sino que trabaja sobre el conjunto de candidatos factibles que ha dejado la revisión anterior.

4.4.6 Justificación de preferencias cumplidas y no solución

Cada candidato conserva un texto de preferencias cumplidas que resume por qué destaca una vez superadas las restricciones. En la salida se muestran también precio, duración, ciudades, días por ciudad, visitas, alojamientos y transportes, de modo que la recomendación no sea una puntuación opaca.

Si no se ha podido elegir ni siquiera el primer plan, el sistema formaliza el resultado como **no solución**. En ese caso se informa de que las restricciones introducidas no se pueden satisfacer con la base actual y se orienta sobre qué condiciones podrían relajarse, como presupuesto, días, avión o calidad mínima.

5 Implementación

5.1 Representación del conocimiento en CLIPS

5.1.1 `deftemplates` y hechos principales

La representación del conocimiento empieza con los `deftemplates`. En ellos definimos la estructura de los hechos que podrán existir en la base de hechos durante la ejecución. Los templates principales son:

- **Fase**, que controla en qué parte del razonamiento está el sistema: bienvenida, preguntas, deducción, generación, selección, salida o fin.
- **Usuario**, que almacena los datos del cliente, restricciones y preferencias.
- **Ciudad**, **Alojamiento**, **Actividad**, **GuiaVisitas** y **Conexion**, que forman la base de conocimiento turística.
- **PlanCandidato**, que representa un itinerario factible generado por el sistema.
- **PlanElegido**, que marca cuáles de los candidatos se van a mostrar como recomendación final.
- **DecisionExperta**, que guarda algunas deducciones relevantes realizadas por las reglas.

Hemos usado valores controlados en muchos slots, por ejemplo en el tipo de grupo, motivo del viaje, calidad de alojamiento, medio de transporte o popularidad de una ciudad. Esto hace que las reglas puedan comparar valores concretos y evita errores por texto libre.

5.1.2 Base de conocimiento de ciudades, alojamientos, visitas y conexiones

La base de conocimiento se carga mediante varios bloques `deffacts`. El primero inicializa el sistema en la fase de bienvenida. Después aparecen los hechos que representan los recursos disponibles.

En **CiudadesDisponibles** se definen destinos con nombre, país, continente, zona, coste diario, seguridad, compatibilidad con niños, popularidad, puntuación base y etiquetas turísticas. Estas etiquetas son las que permiten relacionar una ciudad con motivos como cultura, romanticismo, descanso, diversión o naturaleza.

En **AlojamientosDisponibles** cada alojamiento queda asociado a una ciudad y tiene categoría, calidad numérica, precio por noche y aptitud para niños. En **ActividadesDisponibles** se guardan visitas concretas con tipo, horas, coste, prioridad y compatibilidad familiar. Además, **GuiasDeVisita** agrupa visitas por ciudad y motivo para que la salida final no sea genérica.

Por último, **ConexionesDisponibles** representa transportes desde el origen y entre ciudades. Cada conexión incluye origen, destino, medio, ámbito, coste y horas. Con esto el sistema puede construir rutas multi-ciudad y no solo elegir destinos independientes.

5.1.3 Hechos dinámicos de usuario, candidatos y planes elegidos

Durante la ejecución se crean y modifican hechos dinámicos. El hecho **Usuario** empieza vacío en modo interactivo y se va rellenando con las respuestas de la entrevista. En modo de prueba, los casos automáticos insertan directamente un **Usuario** completo y el sistema salta a las deducciones.

Los **PlanCandidato** se generan solo si una combinación supera los filtros de compatibilidad, días, transporte, alojamiento, guías y presupuesto. Cada candidato ya contiene todo lo necesario para imprimirse: ciudades, días, alojamientos, categorías, transportes, visitas, precio, puntuación y preferencias cumplidas.

Los **PlanElegido** no duplican toda la información del viaje. Solo guardan el orden de recomendación y el identificador del candidato elegido. De esta manera la salida final consulta el candidato original y evita repetir datos en varios hechos.

5.2 Funciones auxiliares del sistema

Las funciones auxiliares se han usado para separar cálculos repetidos de las reglas. Las primeras son **ask-question** y **ask-number**, que validan las respuestas del usuario: una comprueba que la respuesta esté entre los valores permitidos y la otra que el número esté dentro del rango aceptado.

Después hay funciones de normalización, como **calidad-a-num** y **presupuesto-a-nivel**. La primera transforma calidades como *hostal*, *estándar* o *lujo* a una escala numérica. La segunda traduce euros a un nivel orientativo de presupuesto.

El bloque más importante es el de compatibilidad. Ahí se comprueba si el motivo encaja con cada ciudad, si el tipo de guía es válido para el destino, si la calidad del alojamiento cumple el mínimo indicado y si los transportes respetan las restricciones del usuario.

Finalmente están las funciones de puntuación y explicación. Calculan los bonos por tren, ciudades menos conocidas, calidad y precio, y también generan el texto de preferencias cumplidas. Además, se usan funciones específicas para comparar rutas y escoger dos recomendaciones distintas cuando es posible.

5.3 Reglas y fases del motor de inferencia

5.3.1 Bienvenida y entrevista

La ejecución empieza en la fase **bienvenida**. Si no hay ningún **Usuario**, la regla de inicio imprime el encabezado, crea el usuario por defecto y cambia la fase a **preguntas**. A partir de ahí se disparan reglas que preguntan edad, grupo, niños, evento, motivo, presupuesto, duración, número de ciudades, preferencias de transporte, calidad, posibles sacrificios, interés por destinos menos conocidos y ritmo.

La entrevista está organizada como reglas pequeñas. Cada regla solo pregunta un dato si todavía está en valor desconocido o sin inicializar. Cuando todos los campos necesarios están completos, una regla final cambia la fase a **deduccion**. Para los casos automáticos existe una regla especial que detecta un usuario ya cargado y evita repetir la entrevista.

5.3.2 Deducciones expertas

En la fase **deduccion** se aplican reglas que completan o ajustan el perfil del usuario. Primero se calculan valores derivados, como el nivel de presupuesto y el número asociado a la calidad mínima. Luego se aplican deducciones de sentido común: boda o aniversario implican un viaje romántico, fin de curso implica diversión, viajar con niños orienta hacia descanso, pareja sin niños puede ser romántico, amigos jóvenes apuntan a diversión y viajar solo sin motivo se trata como cultural.

También hay ajustes de restricciones. Si el viaje es corto y el usuario había permitido tres ciudades, se limita a dos para evitar una ruta demasiado apretada. Si el presupuesto es económico

y el usuario acepta sacrificar calidad, el sistema permite bajar la calidad mínima de forma controlada.

5.3.3 Generación de candidatos

La generación tiene dos reglas principales: una para rutas de dos ciudades y otra para rutas de tres. Ambas siguen la misma idea. Primero leen el perfil del usuario, después buscan ciudades compatibles con el motivo y la presencia de niños, conexiones de transporte, alojamientos válidos y guías de visita coherentes.

Si todos los recursos encajan, la regla reparte los días entre las ciudades y calcula el precio total. El precio incluye transportes, noches de alojamiento, coste diario de cada ciudad y coste de las visitas. Solo si el precio no supera el presupuesto se calcula la puntuación y se afirma un **PlanCandidato**. Así el sistema no llena la base de hechos con planes que luego habría que descartar.

5.3.4 Selección de recomendaciones

Cuando termina la fase de generación, el sistema pasa a **seleccion**. La primera regla escoge como primer plan el candidato con mayor puntuación. Para el segundo plan se intenta escoger el mejor candidato sin solape de ciudades con el primero.

Si no existe una alternativa sin solape completo, hay una regla de respaldo que busca la mejor ruta distinta. Esta decisión evita imprimir dos veces el mismo viaje con cambios menores, que sería una salida pobre para el enunciado.

5.3.5 Salida explicada y cierre del sistema

En la fase **salida** hay dos caminos. Si no se ha elegido ningún plan, se imprime un mensaje de no solución indicando que las restricciones no pueden satisfacerse con la base actual y sugiriendo relajar presupuesto, días, avión o calidad.

Si hay planes elegidos, el sistema imprime cada recomendación con puntuación interna, precio estimado por persona, duración, ciudades y días por ciudad, visitas, alojamientos, transportes y preferencias cumplidas. Si solo se ha encontrado un plan factible, se avisa de que convendría relajar restricciones para obtener una segunda alternativa. Finalmente se cambia la fase a **fin**.

5.4 Mecanismos de control y modularización del razonamiento

El principal mecanismo de control es el hecho **Fase**. En vez de dejar que todas las reglas puedan activarse en cualquier momento, cada grupo de reglas exige estar en una fase concreta. Esto hace que el razonamiento avance en orden: entrevista, deducción, generación, selección y salida.

También se usan prioridades con **declare (salience ...)**. Las reglas de generación tienen prioridad positiva para crear candidatos antes de pasar a selección. Las reglas de transición tienen salience negativa para ejecutarse cuando ya no quedan reglas más específicas de esa fase. En selección, el primer plan y el segundo sin solape tienen más prioridad que el respaldo con ruta distinta.

La modularización no se ha hecho con varios archivos, sino con secciones claras dentro del mismo **viajes.clp**. Esto nos ha parecido suficiente para el tamaño de la práctica y facilita cargar el sistema con una sola orden.

5.5 Decisiones de implementación relevantes

5.5.1 Restricciones obligatorias frente a preferencias

Una decisión importante ha sido tratar de forma diferente restricciones y preferencias. Las restricciones aparecen como condiciones que un candidato debe cumplir: presupuesto, días, número de ciudades, evitar avión, calidad mínima y compatibilidad familiar. Si fallan, el plan ni siquiera se afirma como `PlanCandidato`.

Las preferencias se aplican después mediante puntuación. Por ejemplo, usar tren, incluir ciudades menos masificadas, superar la calidad mínima o dejar margen de presupuesto suma puntos, pero no invalida por sí solo un plan que cumple las restricciones. Esta separación hace que el sistema sea menos frágil y permita comparar alternativas razonables.

5.5.2 Cálculo numérico de precios

Hemos usado precios numéricos porque el enunciado pide mostrar una recomendación concreta y porque el presupuesto es una restricción dura. Cada candidato suma el coste de los transportes, el coste de alojamiento por noche, el coste diario de cada ciudad y el coste de las guías de visita.

El cálculo se hace dentro de las reglas de generación, justo antes de afirmar el plan. Esto evita generar planes que ya se sabe que superan el presupuesto. Además, el margen restante se usa como parte de la puntuación con `bonus-precio`, de manera que dos planes válidos pueden compararse teniendo en cuenta cuál aprovecha mejor el presupuesto.

5.5.3 Control de rutas distintas y caso imposible

Para cumplir la idea de dos recomendaciones distintas, el identificador del plan incluye ciudades, alojamientos, transportes y guías de visita. Además, la selección del segundo plan usa funciones específicas para comprobar si dos rutas no comparten ciudades o, como mínimo, si no son exactamente la misma.

El caso imposible se controla en la salida. Si después de generar y seleccionar no existe ningún `PlanElegido` de orden 1, se dispara la regla de no solución. Esto es mejor que forzar un viaje incorrecto, porque mantiene la coherencia entre restricciones y recomendación.

5.6 Desarrollo incremental y prototipos

5.6.1 Prototipo inicial

El primer objetivo fue tener un prototipo pequeño pero completo: preguntar datos del usuario, deducir un perfil básico y producir alguna recomendación. En esta fase lo importante no era tener toda la base de conocimiento, sino comprobar que CLIPS podía encadenar preguntas, hechos de usuario y reglas de recomendación.

Esa idea se conserva en la versión final con la fase de entrevista, las reglas de deducción y el control por `Fase`. Aunque el sistema final es más grande, sigue teniendo el mismo esqueleto que un prototipo funcional.

5.6.2 Extensiones y revisiones intermedias

Después ampliamos el prototipo para cubrir mejor lo que pedía el enunciado. Las extensiones principales fueron:

- pasar de destinos sueltos a itinerarios de dos o tres ciudades;
- añadir transporte entre etapas y vuelta al origen;
- incorporar alojamientos, visitas concretas y guías de visita;
- calcular precios numéricos;
- separar restricciones de preferencias;
- seleccionar dos planes diferentes;
- incluir casos de no solución y juegos de prueba automáticos.

Estas revisiones hicieron que el sistema dejara de ser una recomendación simple y pasara a generar paquetes de viaje completos.

5.6.3 Versión final

La versión final integra todos los bloques en un único flujo de razonamiento. El usuario puede usar el sistema de forma interactiva o ejecutar directamente un caso de prueba. En ambos modos se llega a la misma secuencia: deducción del perfil, generación de candidatos, selección de recomendaciones y salida explicada.

El resultado final cumple los puntos principales que necesitábamos: rutas de varias ciudades, transportes entre etapas, precio total, visitas concretas, alojamientos, dos alternativas diferentes cuando sea posible y mensaje de no solución cuando las restricciones son incompatibles.

5.6.4 Planificación y organización del trabajo del grupo

Nos hemos organizado dividiendo el trabajo por partes del sistema. Primero se definió el dominio y la ontología, después se implementó la estructura en CLIPS y finalmente se completaron reglas, base de conocimiento y pruebas. Esta separación nos permitió avanzar sin mezclar todo a la vez.

5.7 Instrucciones de ejecución

Para ejecutar el sistema en modo interactivo desde CLIPS:

```
(clear)
(load "viajes.clp")
(reset)
(run)
```

También se pueden ejecutar los juegos de prueba definidos al final del archivo:

```
(clear)
(load "viajes.clp")
(caso-familia-alto)
(caso-estudiante-tren)
(caso-pareja-romantica)
(caso-amigos-diversion)
(caso-naturaleza-menos-conocido)
(caso-imposible-sin-avion-asia)
```

Estas pruebas no son una ejecución interactiva en la que vamos respondiendo por teclado. Son funciones ya preparadas al final de `viajes.clp` que hacen `reset`, insertan un usuario con las respuestas ya cargadas y después ejecutan `run`. A efectos del razonamiento actúan igual que si ese usuario hubiera contestado la entrevista completa, porque se activa el mismo motor de inferencia, las mismas reglas y la misma salida final.

6 Juegos de prueba y validación

6.1 Criterios de selección de los juegos de prueba

Los juegos de prueba se han escogido para comprobar que el sistema no funciona solo con un caso favorable, sino con perfiles distintos y restricciones diferentes. Hemos intentado cubrir viajes familiares, viajes con presupuesto ajustado, deducciones de perfil, preferencias de transporte, rutas de ocio, viajes de naturaleza y un caso imposible.

Cada prueba se ejecuta con una función definida al final de `viajes.clp`. No son casos interactivos escritos a mano durante la ejecución, sino funciones que simulan esa interacción: hacen `reset`, insertan un usuario ya completo y ejecutan `run`. De esta forma se prueba exactamente el mismo motor de inferencia que en modo interactivo, pero sin tener que contestar manualmente a todas las preguntas cada vez.

6.2 Cobertura de casos comunes, restricciones y excepciones

El conjunto de pruebas cubre los siguientes aspectos del sistema:

- Recomendación para una familia con niños y calidad alta.
- Restricción dura de evitar avión y preferencia por tren.
- Deducción de viaje romántico a partir de un aniversario.
- Deducción de diversión a partir de un fin de curso.
- Preferencia por naturaleza y lugares menos conocidos.
- Detección del caso sin recomendación factible.

La salida completa de las pruebas se conserva en `SALIDAS_PRUEBAS.txt`. En este apartado recogemos los datos principales de esa salida para justificar el comportamiento del sistema.

6.3 Caso 1: familia con niños y presupuesto alto

6.3.1 Escenario, objetivo y entrada

Este caso representa una familia con niños, presupuesto alto y preferencia por un viaje de descanso. El usuario pide entre dos y tres ciudades, ocho días de duración, calidad mínima superior y no impone restricciones sobre el avión.

El objetivo de la prueba es comprobar que el sistema respeta la compatibilidad familiar, mantiene alojamientos de calidad alta y genera un itinerario multi-ciudad dentro del presupuesto.

Comando ejecutado:

```
(caso-familia-alto)
```

6.3.2 Resultado esperado

Esperamos obtener dos planes factibles de descanso, con ciudades y alojamientos aptos para niños, precio inferior a 4500 euros por persona y visitas coherentes con un viaje familiar o relajado.

6.3.3 Salida original de CLIPS

La salida genera dos recomendaciones. Los valores principales fueron:

Plan 1: Punta Cana, Cancun, Costa Rica
Precio: 3488 euros
Duracion: 8 dias
Puntuacion: 237
Preferencias: respeta la calidad minima de alojamiento

Plan 2: Cancun, Costa Rica
Precio: 3222 euros
Duracion: 8 dias
Puntuacion: 169
Preferencias: respeta la calidad minima de alojamiento

6.3.4 Explicación del resultado obtenido

El resultado es correcto porque ambos planes quedan por debajo del presupuesto y usan alojamientos familiares de categoría alta, como resorts o hoteles superiores. Además, el sistema propone destinos de playa, descanso y naturaleza, que encajan con el motivo del viaje y con el perfil familiar. La primera recomendación tiene más puntuación porque incluye tres ciudades y combina más recursos compatibles.

6.4 Caso 2: estudiante que prioriza tren y evita avión

6.4.1 Escenario, objetivo y entrada

Este caso representa un usuario joven que viaja solo, con presupuesto ajustado, motivo cultural, preferencia por tren y restricción explícita de evitar avión. La duración es de seis días y el viaje debe tener dos ciudades.

El objetivo es comprobar que evitar avión actúa como restricción dura y que la preferencia por tren se refleja en la puntuación y en la explicación final.

Comando ejecutado:

(caso-estudiante-tren)

6.4.2 Resultado esperado

Esperamos que no aparezca ningún tramo en avión, que los planes usen transporte ferroviario y que el presupuesto de 1250 euros sea suficiente para generar rutas culturales con alojamientos económicos.

6.4.3 Salida original de CLIPS

La salida genera dos planes dentro del presupuesto:

Plan 1: Lisboa, Sevilla
Precio: 914 euros
Duracion: 6 dias
Puntuacion: 189
Transportes: tren, tren, tren

Plan 2: Paris, Amsterdam
Precio: 1075 euros
Duracion: 6 dias
Puntuacion: 187
Transportes: tren, tren, tren

6.4.4 Explicación del resultado obtenido

El sistema cumple la restricción principal porque todos los transportes son en tren. También respeta el presupuesto y propone visitas culturales en todas las ciudades. El primer plan queda ligeramente por encima en puntuación porque, además de usar tren, incluye ciudades menos masificadas y deja más margen de presupuesto.

6.5 Caso 3: pareja romántica por aniversario

6.5.1 Escenario, objetivo y entrada

Este caso representa una pareja sin niños que viaja por aniversario. El motivo se deja como desconocido para comprobar si el sistema deduce correctamente que el perfil debe ser romántico. El presupuesto es de 2200 euros, la duración es de siete días y se permiten entre dos y tres ciudades.

Comando ejecutado:

(caso-pareja-romantica)

6.5.2 Resultado esperado

Esperamos que la fase de deducción interprete el aniversario como viaje romántico y que las rutas propuestas incluyan ciudades y guías compatibles con ese motivo. También esperamos que se respete el presupuesto y que la preferencia por tren se tenga en cuenta cuando sea posible.

6.5.3 Salida original de CLIPS

La salida confirma dos recomendaciones románticas:

Plan 1: Paris, Roma, Viena
Precio: 1899 euros
Duracion: 7 dias
Puntuacion: 288
Visitas: Torre Eiffel, Trastevere, Belvedere

Plan 2: Lisboa, Sevilla
Precio: 990 euros
Duracion: 7 dias
Puntuacion: 196
Visitas: Alfama, Santa Cruz, Guadalquivir

6.5.4 Explicación del resultado obtenido

El comportamiento es el esperado porque el sistema deduce el motivo romántico a partir del aniversario y selecciona guías coherentes con ese perfil. El primer plan obtiene una puntuación mucho mayor porque combina tres ciudades con alta puntuación base y varias etapas compatibles. El segundo plan ofrece una alternativa más cercana y barata, también dentro del presupuesto.

6.6 Caso 4: grupo de amigos en viaje de fin de curso

6.6.1 Escenario, objetivo y entrada

Este caso representa un grupo de amigos joven con evento de fin de curso. El motivo se deja como desconocido para que el sistema deduzca diversión. El viaje tiene seis días, dos ciudades, calidad mínima estándar y presupuesto de 1700 euros.

Comando ejecutado:

(caso-amigos-diversion)

6.6.2 Resultado esperado

Esperamos que el sistema deduzca un perfil de ocio y diversión, proponga ciudades con actividades nocturnas o de ambiente joven y mantenga el precio por debajo del presupuesto.

6.6.3 Salida original de CLIPS

La salida genera dos alternativas de ocio:

Plan 1: Lisboa, Ibiza
Precio: 1280 euros
Duracion: 6 dias
Puntuacion: 159
Visitas: Barrio Alto, LX Factory, Cala Comte, Dalt Vila

Plan 2: Lisboa, Sevilla
Precio: 909 euros
Duracion: 6 dias
Puntuacion: 157
Visitas: Barrio Alto, LX Factory, Triana, Alameda

6.6.4 Explicación del resultado obtenido

La prueba valida la deducción de diversión a partir del fin de curso. Los dos planes incluyen actividades de ocio y ciudades con etiquetas compatibles. En este caso las dos alternativas

comparten Lisboa, pero no son el mismo recorrido: el sistema recurre a una ruta distinta cuando no encuentra una alternativa mejor sin solape completo.

6.7 Caso 5: naturaleza y lugares menos conocidos

6.7.1 Escenario, objetivo y entrada

Este caso representa una pareja que busca naturaleza, acepta avión y prefiere lugares menos conocidos. El presupuesto es de 3500 euros, la duración de ocho días y se permiten dos o tres ciudades.

Comando ejecutado:

(caso-naturaleza-menos-conocido)

6.7.2 Resultado esperado

Esperamos planes con destinos de naturaleza, visitas relacionadas con ese motivo y bonificación por incluir lugares menos masificados. El precio total debe mantenerse dentro del presupuesto.

6.7.3 Salida original de CLIPS

La salida obtiene dos rutas compatibles:

Plan 1: Kyoto, Bali

Precio: 3442 euros

Duracion: 8 dias

Puntuacion: 178

Visitas: Arashiyama, jardines de templos, Ubud, arrozales

Plan 2: Cancun, Costa Rica

Precio: 2938 euros

Duracion: 8 dias

Puntuacion: 175

Visitas: Playa Delfines, Volcan Arenal, Monteverde

6.7.4 Explicación del resultado obtenido

El resultado es coherente porque las ciudades propuestas tienen etiquetas o guías compatibles con naturaleza y descanso. Además, ambas salidas indican que se cumple la preferencia por ciudades menos masificadas y la calidad mínima del alojamiento. La primera recomendación queda más cerca del límite de presupuesto, pero sigue siendo factible.

6.8 Caso 6: restricciones incompatibles y no solución

6.8.1 Escenario, objetivo y entrada

Este caso fuerza una situación complicada: pareja en viaje de boda, motivo romántico, solo cuatro días, presupuesto de 1200 euros, preferencia por tren, restricción de evitar avión, calidad mínima de lujo y sin aceptar sacrificar calidad ni duración.

Comando ejecutado:

(caso-imposible-sin-avion-asia)

6.8.2 Resultado esperado

Esperamos que no se genere ninguna recomendación factible. Las condiciones son demasiado restrictivas para la base de conocimiento: pocos días, presupuesto bajo, calidad de lujo, dos o tres ciudades y prohibición de avión.

6.8.3 Salida original de CLIPS

La salida confirma el caso de no solución:

NO SE HA ENCONTRADO NINGUNA RECOMENDACION FACTIBLE

Las restricciones introducidas no pueden satisfacerse con la base de conocimiento actual.

Sugerencias: aumentar presupuesto/días, permitir avion o bajar la calidad minima.

6.8.4 Explicación del resultado obtenido

La prueba valida que el sistema no fuerza una recomendación incorrecta. Si no existe ningún candidato que cumpla todas las restricciones, se informa de forma clara y se indican posibles condiciones que el usuario podría relajar.

6.9 Verificación del cumplimiento del enunciado

Con los seis casos se comprueba que el sistema cumple los requisitos principales del enunciado. El sistema pregunta o recibe datos del usuario, deduce perfiles cuando falta información, genera viajes de varias ciudades, incluye alojamientos, visitas y transportes, calcula precio total, devuelve dos planes cuando existen alternativas y contempla el caso sin solución.

También se verifica la diferencia entre restricciones y preferencias. Evitar avión, presupuesto, días, calidad mínima y compatibilidad familiar actúan como condiciones obligatorias. En cambio, usar tren, preferir ciudades menos conocidas o mejorar calidad se usan para puntuar y explicar planes factibles.

6.10 Análisis global de los resultados de prueba

Los resultados son consistentes con el funcionamiento esperado. En los casos factibles se obtienen rutas completas, siempre con precio, duración, ciudades, visitas, alojamientos, transportes y preferencias cumplidas. En el caso imposible no se imprime una ruta artificial, sino un mensaje explícito de no recomendación.

La prueba de amigos muestra una limitación razonable del sistema: cuando no hay suficiente variedad sin solape completo, la segunda alternativa puede compartir alguna ciudad con la primera. Aun así, no se devuelve el mismo plan, sino una ruta distinta. Esto mantiene la salida dentro de la base disponible y evita inventar recursos que no existen.

7 Conclusiones

7.1 Cumplimiento de objetivos

El sistema final cumple los objetivos principales de la práctica. Hemos construido un Sistema Basado en el Conocimiento capaz de recomendar viajes personalizados a partir de datos del usuario, restricciones y preferencias. La solución no se limita a elegir una ciudad, sino que genera itinerarios con dos o tres destinos, transportes, alojamientos, visitas, duración, precio y una explicación de las preferencias cumplidas.

También se cumple el requisito de ofrecer dos alternativas cuando la base de conocimiento lo permite. Además, el sistema contempla el caso en que no existe ninguna combinación factible y lo comunica de forma explícita.

7.2 Fortalezas del sistema construido

Una de las fortalezas del sistema es que separa bien el conocimiento del dominio del razonamiento. Las ciudades, alojamientos, visitas y conexiones están representadas como hechos, mientras que las decisiones de experto aparecen en reglas y funciones de compatibilidad.

Otra fortaleza es la explicabilidad. La salida no muestra solo un nombre de destino, sino el precio, los días, los transportes, las visitas, los alojamientos y las preferencias satisfechas. Esto hace que el usuario pueda entender por qué se le recomienda cada plan.

También consideramos positiva la modularización por fases. El hecho **Fase** permite que la entrevista, la deducción, la generación, la selección y la salida se ejecuten en orden y sean más fáciles de revisar.

7.3 Limitaciones actuales y posibles mejoras

La principal limitación es que la base de conocimiento es cerrada. El sistema no consulta disponibilidad real de vuelos, hoteles o actividades, y los precios son estimaciones usadas para razonar dentro de la práctica. En una versión más completa se podrían conectar fuentes externas o actualizar precios según temporada.

Otra limitación es que el origen se modela de forma genérica. Esto simplifica la práctica, pero en un sistema real el origen del usuario afectaría mucho al precio y al transporte. También podrían incorporarse fechas, clima, visados, horarios y preferencias más finas sobre ritmo o tipo de alojamiento.

Como mejora del razonamiento, se podría enriquecer la explicación final con las decisiones expertas deducidas durante la ejecución, por ejemplo indicando que un aniversario se ha interpretado como viaje romántico o que un viaje corto ha limitado el número de ciudades.

7.4 Aprendizajes sobre ontologías, reglas y Sistemas Basados en el Conocimiento

La práctica nos ha servido para ver que construir un SBC no consiste solo en programar reglas sueltas. Primero hay que entender el dominio, decidir qué conceptos importan, separar restricciones de preferencias y formalizar una ontología que después pueda trasladarse a hechos y reglas.

También hemos visto la importancia de probar casos variados. Un sistema puede parecer correcto con un perfil sencillo, pero fallar cuando se combinan restricciones fuertes. Por eso los juegos de prueba han sido útiles para comprobar tanto recomendaciones normales como el caso sin solución.

Finalmente, hemos aprendido que CLIPS encaja bien cuando el problema se puede expresar mediante hechos, condiciones y reglas de inferencia. En nuestro caso, el enfoque nos ha permitido representar conocimiento explícito y justificar las recomendaciones de una forma bastante clara.

A Salidas literales completas de los juegos de prueba

Este apéndice recoge las salidas completas guardadas en `SALIDAS_PRUEBAS.txt`. En el apartado 6 hemos usado un resumen de cada caso para no cortar el razonamiento principal de la memoria; aquí dejamos la evidencia literal que permite comprobar que las pruebas se han ejecutado sobre el sistema final. Todas estas salidas vienen de funciones de prueba definidas al final de `viajes.clp`. Esas funciones precargan el usuario y lanzan `run`, por lo que no requieren entrevista manual, pero recorren el mismo razonamiento que una ejecución interactiva.

A.1 Preparación de la ejecución

```
SALIDAS LITERALES DE LOS JUEGOS DE PRUEBA
Sistema: viajes.clp
Comandos previos en CLIPS:
  (clear)
  (load "viajes.clp")
=====
```

A.2 Caso familia

```
CASO 1: Familia con niños y presupuesto alto
Comando: (caso-familia-alto)
Salida literal:
-> [MODO PRUEBA] Usuario precargado. Iniciando deducciones...
-> [INFO] Perfil deducido. Generando itinerarios candidatos...
-> [INFO] Seleccionando las mejores alternativas...

=====
PLAN 2 RECOMENDADO
=====
Puntuacion interna: 169
Precio total estimado por persona: 3222 euros
Duracion: 8 dias
Ciudades: Cancun (4 dias), Costa Rica (4 dias)
Visitas:
- Cancun: Playa Delfines, cenote, tarde libre
- Costa Rica: Volcan Arenal, Monteverde, sendero nocturno
Alojamiento:
- Cancun: Resort Maya Familiar (resort)
- Costa Rica: Lodge Arenal 4* (hotel-superior)
Transportes:
- Origen -> Cancun: avion
- Cancun -> Costa Rica: avion
- Costa Rica -> Origen: avion
Preferencias cumplidas: respeta la calidad minima de alojamiento;

=====
PLAN 1 RECOMENDADO
=====
Puntuacion interna: 237
Precio total estimado por persona: 3488 euros
Duracion: 8 dias
Ciudades: Punta Cana (2 dias), Cancun (3 dias), Costa Rica (3 dias)
Visitas:
- Punta Cana: Playa Bavaro, Isla Saona, tarde libre en resort
- Cancun: Playa Delfines, cenote, tarde libre
```

```

- Costa Rica: Volcan Arenal, Monteverde, sendero nocturno
Alojamiento:
- Punta Cana: Resort Bavaro Familiar (resort)
- Cancun: Resort Maya Familiar (resort)
- Costa Rica: Lodge Arenal 4* (hotel-superior)
Transportes:
- Origen -> Punta Cana: avion
- Punta Cana -> Cancun: avion
- Cancun -> Costa Rica: avion
- Costa Rica -> Origen: avion
Preferencias cumplidas: respeta la calidad minima de alojamiento;

-> [FIN] Razonamiento completado.

```

A.3 Caso estudiante

```

=====
CASO 2: Estudiante que prioriza tren y evita avion
Comando: (caso-estudiante-tren)
Salida literal:
-> [MOD0 PRUEBA] Usuario precargado. Iniciando deducciones...
-> [INFO] Perfil deducido. Generando itinerarios candidatos...
-> [INFO] Seleccionando las mejores alternativas...

=====
PLAN 2 RECOMENDADO
=====
Puntuacion interna: 187
Precio total estimado por persona: 1075 euros
Duracion: 6 dias
Ciudades: Paris (3 dias), Amsterdam (3 dias)
Visitas:
- Paris: Louvre, Torre Eiffel, paseo por el Sena
- Amsterdam: Rijksmuseum, canales, Jordaan
Alojamiento:
- Paris: Hostal Montmartre (hostal)
- Amsterdam: Hostal Canal (hostal)
Transportes:
- Origen -> Paris: tren
- Paris -> Amsterdam: tren
- Amsterdam -> Origen: tren
Preferencias cumplidas: usa tren en algun tramo; respeta la calidad minima de alojamiento;

=====
PLAN 1 RECOMENDADO
=====
Puntuacion interna: 189
Precio total estimado por persona: 914 euros
Duracion: 6 dias
Ciudades: Lisboa (3 dias), Sevilla (3 dias)
Visitas:
- Lisboa: Belem, Jeronimos, Museo del Azulejo
- Sevilla: Real Alcazar, Catedral, Archivo de Indias
Alojamiento:
- Lisboa: Apartamento Alfama (apartamento)
- Sevilla: Apartamento Santa Cruz (apartamento)
Transportes:
- Origen -> Lisboa: tren
- Lisboa -> Sevilla: tren
- Sevilla -> Origen: tren
Preferencias cumplidas: usa tren en algun tramo; incluye ciudades menos masificadas; respeta la calidad
minima de alojamiento;

-> [FIN] Razonamiento completado.

```

A.4 Caso pareja

```

=====
CASO 3: Pareja romantica por aniversario
Comando: (caso-pareja-romantica)
Salida literal:
-> [MODO PRUEBA] Usuario precargado. Iniciando deducciones...
-> [INFO] Perfil deducido. Generando itinerarios candidatos...
-> [INFO] Seleccionando las mejores alternativas...

=====
PLAN 2 RECOMENDADO
=====
Puntuacion interna: 196
Precio total estimado por persona: 990 euros
Duracion: 7 dias
Ciudades: Lisboa (3 dias), Sevilla (4 dias)
Visitas:
- Lisboa: Alfama, mirador de Santa Lucia, tranvia 28
- Sevilla: Real Alcazar, Santa Cruz, atardecer junto al Guadalquivir
Alojamiento:
- Lisboa: Apartamento Alfama (apartamento)
- Sevilla: Apartamento Santa Cruz (apartamento)
Transportes:
- Origen -> Lisboa: tren
- Lisboa -> Sevilla: tren
- Sevilla -> Origen: tren
Preferencias cumplidas: usa tren en algun tramo; incluye ciudades menos masificadas; respeta la calidad minima de alojamiento;

=====
PLAN 1 RECOMENDADO
=====
Puntuacion interna: 288
Precio total estimado por persona: 1899 euros
Duracion: 7 dias
Ciudades: Paris (2 dias), Roma (2 dias), Viena (3 dias)
Visitas:
- Paris: Torre Eiffel, Montmartre, paseo nocturno por el Sena
- Roma: Trastevere, Fontana di Trevi, mirador del Gianicolo
- Viena: Belvedere, cafe historico, paseo por el Danubio
Alojamiento:
- Paris: Hotel Seine 3* (hotel-estandar)
- Roma: Hotel Navona 4* (hotel-superior)
- Viena: Hotel Opera 4* (hotel-superior)
Transportes:
- Origen -> Paris: tren
- Paris -> Roma: tren
- Roma -> Viena: tren
- Viena -> Origen: avion
Preferencias cumplidas: usa tren en algun tramo; incluye ciudades menos masificadas; respeta la calidad minima de alojamiento;

-> [FIN] Razonamiento completado.

```

A.5 Caso amigos

```

=====
CASO 4: Amigos en viaje de fin de curso
Comando: (caso-amigos-diversion)
Salida literal:
-> [MODO PRUEBA] Usuario precargado. Iniciando deducciones...
-> [INFO] Perfil deducido. Generando itinerarios candidatos...
-> [INFO] Seleccionando las mejores alternativas...

=====
PLAN 2 RECOMENDADO
=====
Puntuacion interna: 157
Precio total estimado por persona: 909 euros
Duracion: 6 dias

```

```

Ciudades: Lisboa (3 dias), Sevilla (3 dias)
Visitas:
- Lisboa: Barrio Alto, LX Factory, tarde en Cais do Sodre
- Sevilla: Triana, Alameda de Hercules, noche en el centro
Alojamiento:
- Lisboa: Apartamento Alfama (apartamento)
- Sevilla: Apartamento Santa Cruz (apartamento)
Transportes:
- Origen -> Lisboa: tren
- Lisboa -> Sevilla: tren
- Sevilla -> Origen: tren
Preferencias cumplidas: respeta la calidad minima de alojamiento;

=====
PLAN 1 RECOMENDADO
=====
Puntuacion interna: 159
Precio total estimado por persona: 1280 euros
Duracion: 6 dias
Ciudades: Lisboa (3 dias), Ibiza (3 dias)
Visitas:
- Lisboa: Barrio Alto, LX Factory, tarde en Cais do Sodre
- Ibiza: Cala Comte, Dalt Vila, noche en Playa d'en Bossa
Alojamiento:
- Lisboa: Apartamento Alfama (apartamento)
- Ibiza: Hotel Puerto 3* (hotel-estandar)
Transportes:
- Origen -> Lisboa: tren
- Lisboa -> Ibiza: avion
- Ibiza -> Origen: barco
Preferencias cumplidas: respeta la calidad minima de alojamiento;

-> [FIN] Razonamiento completado.

```

A.6 Caso naturaleza

```

=====
CASO 5: Naturaleza y lugares menos conocidos
Comando: (caso-naturaleza-menos-conocido)
Salida literal:
-> [MOD0 PRUEBA] Usuario precargado. Iniciando deducciones...
-> [INFO] Perfil deducido. Generando itinerarios candidatos...
-> [INFO] Seleccionando las mejores alternativas...

=====
PLAN 2 RECOMENDADO
=====
Puntuacion interna: 175
Precio total estimado por persona: 2938 euros
Duracion: 8 dias
Ciudades: Cancun (4 dias), Costa Rica (4 dias)
Visitas:
- Cancun: Playa Delfines, cenote, tarde libre
- Costa Rica: Volcan Arenal, Monteverde, sendero nocturno
Alojamiento:
- Cancun: Hotel Centro 3* (hotel-estandar)
- Costa Rica: Lodge Arenal 4* (hotel-superior)
Transportes:
- Origen -> Cancun: avion
- Cancun -> Costa Rica: avion
- Costa Rica -> Origen: avion
Preferencias cumplidas: incluye ciudades menos masificadas; respeta la calidad minima de alojamiento;

=====
PLAN 1 RECOMENDADO
=====
Puntuacion interna: 178
Precio total estimado por persona: 3442 euros
Duracion: 8 dias
Ciudades: Kyoto (4 dias), Bali (4 dias)

```

```

Visitas:
- Kyoto: Arashiyama, jardines de templos, paseo por el rio Kamo
- Bali: Ubud, arrozales de Tegallalang, playa de Sanur
Alojamiento:
- Kyoto: Ryokan Higashiyama (hotel-superior)
- Bali: Resort Ubud (resort)
Transportes:
- Origen -> Kyoto: avion
- Kyoto -> Bali: avion
- Bali -> Origen: avion
Preferencias cumplidas: incluye ciudades menos masificadas; respeta la calidad minima de alojamiento;

-> [FIN] Razonamiento completado.

```

A.7 Caso sin solución

```

=====
CASO 6: Restricciones incompatibles
Comando: (caso-imposible-sin-avion-asia)
Salida literal:
-> [MODULO PRUEBA] Usuario precargado. Iniciando deducciones...
-> [INFO] Perfil deducido. Generando itinerarios candidatos...
-> [INFO] Seleccionando las mejores alternativas...

=====
NO SE HA ENCONTRADO NINGUNA RECOMENDACION FACTIBLE
=====
Las restricciones introducidas no pueden satisfacerse con la base de conocimiento actual.
Sugerencias: aumentar presupuesto/dias, permitir avion o bajar la calidad minima.

-> [FIN] Razonamiento completado.

```

B Documentación de la ontología generada con Protégé

B.1 Archivo y alcance de la ontología

La ontología se entrega en el archivo `ontologia_viajes.ttl`. Está en formato Turtle y se puede abrir directamente con Protégé. Su objetivo es documentar el vocabulario conceptual del sistema: las clases principales del dominio, las propiedades que relacionan los conceptos y los atributos más importantes que después se usan en CLIPS.

Hemos separado la ontología del motor de inferencia porque cada parte tiene una función distinta. La ontología sirve para dejar claro qué conceptos existen y cómo se relacionan. El archivo `viajes.clp`, en cambio, materializa esa conceptualización en hechos y reglas ejecutables.

B.2 Grafo de clases y relaciones

La jerarquía de clases de la ontología parte de una clase raíz **Entidad**. De ella cuelgan los conceptos principales del dominio: **Usuario**, **Ciudad**, **Servicio**, **Restriccion**, **Preferencia**, **PaqueteViaje** y **Eta**. Dentro de **Servicio** hemos agrupado los recursos concretos que puede utilizar un viaje: **Alojamiento**, **Transporte** y **Actividad**.

Origen	Relación	Destino
PaqueteViaje	viajaA	Ciudad
PaqueteViaje	incluyeEtapa	Etapa
PaqueteViaje	usaTransporte	Transporte
Etapa	usaAlojamiento	Alojamiento
Etapa	incluyeActividad	Actividad
PaqueteViaje	cumplePreferencia	Preferencia
PaqueteViaje	respetaRestriccion	Restriccion

Estas relaciones reflejan la estructura real de la recomendación. Un paquete de viaje no contiene solo un destino, sino varias etapas, transportes, alojamientos, actividades y criterios de evaluación.

B.3 Fichas de clases, propiedades e instancias

B.3.1 Clases principales

- **Usuario:** representa el perfil del cliente. Incluye edad, grupo, presencia de niños, presupuesto, duración, motivo, calidad mínima y preferencias.
- **Ciudad:** representa cada destino candidato. Guarda nombre, país, continente, zona, coste diario, seguridad, compatibilidad familiar, popularidad y etiquetas turísticas.
- **Servicio:** clase general para recursos usados por el viaje. Sus subclases son **Alojamiento**, **Transporte** y **Actividad**.
- **Alojamiento:** recurso de estancia asociado a una ciudad, con categoría, calidad, precio por noche y compatibilidad con niños.
- **Transporte:** conexión utilizada para llegar a los destinos, moverse entre ciudades o volver al origen.
- **Actividad:** visita o experiencia concreta de una ciudad.
- **Restriccion:** condición obligatoria que un plan debe cumplir. En el sistema aparecen como presupuesto, duración, evitar avión, número de ciudades o calidad mínima.
- **Preferencia:** criterio usado para comparar soluciones factibles, por ejemplo usar tren, escoger lugares menos conocidos o mejorar la calidad.
- **PaqueteViaje:** solución recomendada, con ciudades, etapas, precio total y puntuación.
- **Etapa:** parte de un paquete centrada en una ciudad concreta.

B.3.2 Propiedades de objeto

Las propiedades de objeto indican cómo se conectan las clases. La ontología define **viajaA**, **incluyeEtapa**, **usaAlojamiento**, **incluyeActividad**, **usaTransporte**, **cumplePreferencia** y **respetaRestriccion**. Con ellas se puede expresar que una recomendación visita ciudades, se compone de etapas, usa recursos y se evalúa con restricciones y preferencias.

B.3.3 Propiedades de datos

Las propiedades de datos recogen los atributos numéricos o textuales necesarios para razonar. En `Usuario` aparecen `edad`, `presupuestoEuros` y `duracionMaxDias`. En `Ciudad` aparece `costeDiario`. En `Alojamiento` se usan `precioNoche` y `calidad`. En `Transporte` se usan `costeTransporte` y `horasTransporte`. Finalmente, `PaqueteViaje` tiene `precioTotal` y `puntuacion`.

B.3.4 Instancias representadas en la base de conocimiento

El archivo `ontologia_viajes.ttl` documenta clases y propiedades. Las instancias operativas se han representado en `viajes.clp`, porque son las que necesita el motor de inferencia durante la ejecución. La base de conocimiento contiene:

- 16 ciudades en `CiudadesDisponibles`.
- 34 alojamientos en `AlojamientosDisponibles`.
- 32 actividades en `ActividadesDisponibles`.
- 36 guías de visita en `GuiasDeVisita`.
- 62 conexiones de transporte en `ConexionesDisponibles`.

Esta decisión evita duplicar toda la base de datos en dos formatos distintos. La ontología queda como documentación conceptual y CLIPS queda como representación ejecutable del conocimiento.